

Delegated Multi-party Private Set Intersection from Secret Sharing

Jingwei Hu¹, Zhiqi Liu², Cong Zuo², San Ling¹ and Huaxiong Wang¹

¹ Nanyang Technological University, Singapore, {davidhu, lingsan, hxwang}@ntu.edu.sg

² Beijing Institute of Technology, China, {liuzhiqi25, zuocong10}@gmail.com

Abstract. In this work, we address the problem of Delegated PSI (D-PSI), where a cloud server is introduced to handle most computational and communication tasks. D-PSI enables users to securely delegate their private sets to the cloud, ensuring the privacy of their data while allowing efficient computation of the intersection. The cloud operates under strict security requirements, learning nothing about the individual sets or the intersection result. Moreover, D-PSI minimizes user-to-user communication and supports "silent" processing, where the cloud can perform computations independently without user interaction, apart from set delegation and result retrieval.

We formally define the D-PSI problem and propose a novel construction that extends beyond the two-party setting to support multi-party scenarios. Our construction satisfies the D-PSI requirements, provides security against semi-honest adversaries, and can be upgraded to resist malicious behavior. It achieves computational and communication costs close to the ideal perfect D-PSI protocol. We also demonstrate its practicality through a baseline implementation and an optimized variant that further reduces overhead. Our results lay a solid foundation for secure and efficient PSI in real-world cloud environments.

Keywords: Multi-party PSI · Secret Sharing · Randomized Intersection Encoding

1 Introduction

Private Set Intersection (PSI) is arguably one of the most extensively studied secure multiparty computation (MPC) protocols. PSI is essentially a specialized form of MPC, where multiple parties, each holding a private set, collaboratively compute the intersection of their sets. During the computation, no information about the individual sets except for the intersection is leaked to either party.

Early PSI protocols primarily focused on the two-party setting, with classical constructions based on DiffieHellman assumptions [Mea86, HFH99]. In recent years, the focus has shifted toward high-performance designs, where Oblivious Key-Value Stores (OKVS) [GPR⁺21] and Vector Oblivious Linear Evaluation (VOLE) [RS21] serve as core building blocks. In particular, OKVS enables efficient oblivious data encodings, while VOLE facilitates the construction of efficient Oblivious Pseudorandom Functions (OPRF), leading to scalable OPRF-based PSI protocols.

From Two-Party to Multi-Party PSI More recently, the research frontier has moved beyond the two-party setting toward *standard multi-party PSI*, where N parties jointly compute the intersection of their sets. Existing constructions can be broadly categorized into the *semi-honest* and *malicious* models.

In the semi-honest setting, early work [KKRT17] introduced key primitives such as OPRF and zero-sharing, laying the foundation for multi-party PSI. Subsequent works explored more abstract functionalities such as weak private set membership (wPSM) [CGST21]. With the introduction of OKVS, later constructions [WYC24] significantly improved efficiency by combining OPRF and OKVS. Alternative approaches, such as bicentric zero-sharing [GLWS24], reduce multi-party PSI to two-party PSI, though initially relying on additional non-collusion assumptions, which were later removed in [GZWS25].

In the malicious model, stronger security guarantees are required. Representative work [NTY21] leverages OPRF and OKVS to reduce N -party PSI to $(t + 1)$ -party PSI, ensuring security against up to t colluding parties. Other constructions [BEKSS22] based on Oblivious Transfer (OT) and Garbled Bloom Filters (GBF) demonstrate the diversity of feasible approaches under malicious security.

Limitations of Standard Multi-Party PSI Despite these advances, *standard multi-party PSI protocols share a common limitation*: they assume that all parties locally hold their input sets and actively participate in the computation. This assumption becomes increasingly restrictive in modern applications.

In practice, the *cloud computing paradigm is ubiquitous*, where resource-constrained clients rely on powerful cloud servers for storage and computation. Directly applying standard multi-party PSI in such settings leads to several challenges. First, the computation and communication overhead can become prohibitive for clients with limited resources. Second, the protocol requires the full participation of all parties, which limits scalability and flexibility. Third, existing designs do not naturally support outsourced computation, making them ill-suited for cloud-assisted environments.

These limitations motivate the study of *delegated multi-party PSI*, where clients securely outsource their data and computation to a cloud server.

In this paradigm, each party delegates its input set to the cloud, which performs the majority of the computation. However, this introduces new security requirements. In particular, the cloud must learn nothing about the delegated data, the computation must preserve end-to-end privacy guarantees, and the intersection result should remain hidden from the cloud and be revealed only to authorized parties.

This shift from standard to delegated multi-party PSI reflects a fundamental transition:

*from fully local, symmetric computation
to asymmetric, cloud-assisted secure computation.*

Designing efficient and secure protocols under this model remains a challenging and increasingly important problem.

Definition of D-PSI To capture the intuitions above, we formally define the problem of Delegated Private Set Intersection (Delegated PSI, D-PSI) as follows. Unlike the standard multi-party private set intersection problem, D-PSI introduces an additional computing party, a cloud server. Three types of computing parties are defined:

- A cloud server C ,
- Delegated users $P_1, P_2, \dots, P_N$,
- A querying user P_{N+1} .

In this setup, each delegated user P_i securely delegates their set A_i to the cloud server C , and the querying user P_{N+1} also delegates his set A_{N+1} to the cloud and ultimately learns the intersection result. P_{N+1} can be either a new party or one of the delegated users.

The goal of D-PSI is to collaboratively compute the intersection without revealing the individual sets:

- Each delegated user P_i securely (indistinguishable from random values) stores their set on the cloud C .
- The querying user P_{N+1} securely stores their set on the cloud C .
- The cloud C , together with $\{P_i\}_i$ and P_{N+1} , collaboratively computes the intersection such that P_{N+1} can efficiently obtain the result $\cap A_i$.

The requirements are:

- Delegated users $\{P_i\}_{i=1}^N$ only know their private sets.
- The cloud C learns nothing.
- The querying user P_{N+1} only knows their private set A_{N+1} and the intersection $\cap A_i$.
- The cloud C does not collude with the users and a user does not collude with another user.
- The cloud C should handle the majority of the computational and communication tasks.
- Users $\{P_i\}_i$ and P_{N+1} should not interact directly or at least should keep such interaction minimal.
- Silent processing mode is desirable: the cloud can perform the computational tasks independently, without being interrupted by users. This means that users and the cloud can remain offline during the computation and only come online when the user delegates their set or requests the intersection result.

An important feature of D-PSI is its flexibility. If P_{N+1} is one of the delegated users, the protocol degenerates into a standard multi-party private set intersection, where all N parties jointly compute the intersection. If P_{N+1} is a newcomer, the delegated users no longer need to directly interact with him; instead, the cloud performs the computation and engages with the querying user. This design makes D-PSI particularly suitable for cloud-centered computing environments.

A natural question that arises here is: what are the best achievable computation and communication complexities for a D-PSI protocol? We analyze the performance of an ideal "perfect" D-PSI protocol under the assumption that the cloud C is fully trustworthy. This means that all users can directly delegate their sets to the cloud, and the cloud C neither leaks any intermediate results observed during the PSI computation nor the final intersection result. The protocol proceeds as follows:

1. The delegating users $\{P_i\}_i$ upload their respective sets to the cloud C through an encrypted channel.
2. The querying user P_{N+1} also uploads his set to C through an encrypted channel.
3. The cloud C computes the intersection $\cap P_i$ locally and privately.
4. The cloud C returns the intersection $\cap P_i$ to the querying user P_{N+1} through an encrypted channel.

Assume there are N delegating users, each with a set size of k . If no advanced data compression techniques are applied to reduce the size of each user’s set, the communication complexity is clearly:

$$\Omega(Nk).$$

The computational complexity requires a more detailed analysis. Suppose we have a perfect hashing scheme such that a hash table can be used to represent a set, where the hash table has $h = \Omega(k)$ rows, and each row contains at most 1 set element. Such perfect hashing scheme can be realized by a combination of Cuckoo hashing and simple hashing. To compute the intersection, for example $A_1 \cap A_2$, it suffices to compare the set elements in the hash tables HT_{A_1} and HT_{A_2} row by row according to their row correspondence. This results in an intersection hash table $HT_{A_1 \cap A_2}$, which also has h rows and at most k set elements.

Using this technique, the computational complexity of the D-PSI protocol is:

$$O(Nk).$$

Input-Free Query Party Beyond the cloud-assisted setting discussed above, another important yet less explored scenario arises when the party requesting the intersection does not hold any input set.

Consider the following Input-Free Query Party setting [YY25]: the first N parties each hold a private set, while an additional party P_{N+1} acts solely as a *query party*, aiming to learn the intersection of all N sets without contributing any input. In this scenario, standard PSI protocols are not directly applicable, as they inherently assume that any party obtaining the intersection must also provide an input set and actively participate in the computation.

Such a setting naturally appears in real-world applications. For example, a regulatory authority may suspect an individual of financial misconduct and wishes to determine whether this individual appears simultaneously in the credit blacklists of N different banks. However, each bank is unwilling to disclose its entire blacklist and is only willing to reveal the subset of individuals that appear in the intersection across all banks. The regulator itself does not possess any blacklist, and thus naturally acts as a third-party query party.

This example highlights another fundamental limitation of standard multi-party PSI: it tightly couples *data ownership* with *result retrieval*. In many practical scenarios, however, these two roles are inherently decoupled. This observation further motivates the study of more flexible PSI paradigms, where the party learning the intersection may differ from the data providers, and where computation can be delegated to more powerful entities such as the cloud.

Our Contributions. We summarize our main contributions as follows:

1. **Delegated multi-party PSI.** We identify fundamental limitations in existing delegated PSI protocols and propose a new construction that naturally extends to the multi-party setting. Our protocol adheres to the formal notion of delegated PSI, achieving silent preprocessing and requiring only minimal interaction among users.
2. **Enhanced functionality and security.** We show that our construction can be efficiently extended to support dynamic set updates and can be upgraded to achieve malicious security with standard techniques.
3. **Efficiency and optimizations.** We introduce several optimization techniques that may be of independent interest. Our baseline construction achieves $\Omega(Nk^2)$ computational complexity and optimal $\Omega(Nk)$ communication. With further optimizations, the computational complexity is reduced to $\Omega(Nk \log k \log \log k)$ and

the communication complexity to $\Omega(Nk \log k)$. While still not matching the ideal perfect delegated PSI, our protocol significantly improves practical efficiency.

4. **Input-free query support.** We further extend our protocol to support the *input-free query party* setting, where the querying party does not hold any input set. To the best of our knowledge, this is the first delegated multi-party PSI protocol that supports such functionality, thereby overcoming a fundamental limitation of prior approaches.

2 Related works

The research line of private set intersection is extensive and thus we limit our review to the most relevant works related to multi-party private set intersection. Delegated PSI protocols leverage cloud computing for both computation and storage, and they can be categorized into two types: protocols supporting one-off delegation and those enabling repeated delegation for PSI computation.

The one-off delegation protocols, such as those in [Ker12, KMRS14], require clients to locally re-encode their data for each intersection computation and do not allow the reuse of outsourced encrypted data. However, this protocol is designed and implemented solely for a two-party setting.

In contrast, repeated PSI delegation protocols allow clients to outsource their encrypted data to the cloud once and, with the data owner's consent, perform multiple PSI computations without requiring clients to retain a local copy of their data or re-encode it for each computation. Among repeated PSI delegation protocols, [LNZ⁺14] employs hash functions and symmetric key encryption, while [QLS⁺15] utilizes public key encryption and hash functions. The protocol in [ZX15] incorporates verification capabilities using cryptographic accumulators and proxy re-encryption. However, these protocols ([LNZ⁺14, QLS⁺15, ZX15]) are not fully secure and are vulnerable to various attacks, as highlighted in [ATD15, ATMD17]. In contrast, the protocols in [ATD15, ATD17, ATMD17] provide robust security. Protocols in [ATD15, ATD17] use public key encryption and represent the entire dataset as a blinded polynomial that is outsourced to the cloud. Among them, the protocol in [ATMD17] is more efficient, leveraging a hash table to enhance PSI computation performance.

It is important to note that all these secure protocols ([ATD15, ATD17, ATMD17]) are designed and implemented for two-party settings and only support static datasets. A more recent study [ATD20] proposes a new solution to overcome this barrier to support multi-party and dataset updates.

Now let us focus on the recent study [ATD20]. This study explores advanced data structures such as hash tables, bloom filters, and point-evaluation formats to improve the concrete efficiency of the protocol. However, these enhancements make it challenging to provide an overview of their algorithms and, for our purposes, to identify the security flaws in their protocol design.

To address this challenge, we rewrite their protocol in a more concise form, omitting the advanced data structures and the complex set update operations. The core intuition of their algorithm involves setting up a cloud entity C that accepts one secret share of the delegating user's (denoted as A) private set. Due to the security of the secret sharing scheme, the cloud C learns nothing about the delegating user's private set. The crux of computing the intersection between the delegating user A and the querying user B lies in calculating a special encoding, which we refer to as "Randomized Intersection Encoding (RIE)," within the secret sharing format.

2.1 Randomized Intersection Encoding (RIE)

Before delving into the description of the PSI protocol proposed in [ATD20], we provide a brief introduction to the RIE technique. This serves as a foundation for explaining the insecurity we identified in [ATD20], while a formal discussion of RIE is deferred to the Preliminaries section.

A set $A = \{a_i\}_i$ can be equivalently represented as a polynomial:

$$f^{(A)}(x) = \prod_i (x - a_i),$$

which means the elements of the set a_i are encoded as the roots of the polynomial $f^{(A)}(x)$.

Moreover, the intersection of two sets A and B can be expressed as the sum of their corresponding polynomials:

$$f^{(A \cap B)}(x) \stackrel{\text{def}}{=} f^{(A)}(x) + f^{(B)}(x) = \prod_i (x - a_i) + \prod_i (x - b_i).$$

Clearly, the roots of $f^{(A \cap B)}(x)$ represent the intersection $A \cap B$. Either party can determine the intersection $A \cap B$ through a zero-point test.

However, in private set intersection problems, the polynomial $f^{(A \cap B)}(x)$ not only reveals the intersection but can also be used to infer $f^{(A)}(x)$ or $f^{(B)}(x)$, compromising privacy. Therefore, it is essential to construct a polynomial that satisfies two requirements:

- Its roots represent the intersection $A \cap B$.
- It is a randomized polynomial that does not provide additional information to reverse-engineer $f^{(A)}(x)$ or $f^{(B)}(x)$.

A novel method was proposed in [KS05] for the first time to fulfil the two requirements mentioned above, referred to as "Random Intersection Encoding (RIE)":

$$\text{RIE}_{\omega^{(A)}(x), \omega^{(B)}(x)}(f^{(A)}(x), f^{(B)}(x)) \stackrel{\text{def}}{=} \omega^{(A)}(x) \cdot f^{(A)}(x) + \omega^{(B)}(x) \cdot f^{(B)}(x).$$

Privacy Guarantee The RIE encoding process ensures that $f^{(A)}(x)$ and $f^{(B)}(x)$ are not disclosed given that the random polynomials $\omega^{(A)}(x), \omega^{(B)}(x)$ are kept secret. Notably, the RIE encoding output is a "randomized" polynomial that only contains information about the intersection.

Linearity of RIE Encoding RIE encoding is linear. Specifically, for any polynomials $f^{(A)}(x), g^{(A)}(x)$ and $f^{(B)}(x), g^{(B)}(x)$, we have:

$$\begin{aligned} & \text{RIE}_{\omega^{(A)}(x), \omega^{(B)}(x)}(f^{(A)}(x), f^{(B)}(x)) + \text{RIE}_{\omega^{(A)}(x), \omega^{(B)}(x)}(g^{(A)}(x), g^{(B)}(x)) \\ &= \text{RIE}_{\omega^{(A)}(x), \omega^{(B)}(x)}(f^{(A)}(x) + g^{(A)}(x), f^{(B)}(x) + g^{(B)}(x)). \end{aligned}$$

This property ensures that the encoding process preserves the algebraic structure of the input polynomials.

2.2 Case study

Now, let us return to the framework proposed in [ATD20]. For simplicity of notation, we represent a polynomial $p(x)$ directly as p , omitting " (x) ". Assume that we have only one delegated user P_A , and one querying user P_B . The delegated user P_A secret-shares their set polynomial as $([p^{(A)}]_0, [p^{(A)}]_1)$, such that $p^{(A)} = [p^{(A)}]_0 + [p^{(A)}]_1$, and

the querying user P_B also secret-shares their set polynomial as $([p^{(B)}]_0, [p^{(B)}]_1)$. User A delegates one share $[p^{(A)}]_0$ to the cloud while keeping $[p^{(A)}]_1$ secret. Similarly, user P_B delegates $[p^{(B)}]_0$ to the cloud while retaining $[p^{(B)}]_1$ privately.

The PSI protocol proceeds as follows:

1. User P_B splits their private share $[p^{(B)}]_1$ into two parts (o, z) , such that $[p^{(B)}]_1 = o + z$.
2. User P_B sends o to the cloud and z to user P_A .
3. User P_A generates two random polynomials $\omega^{(A)}$ and $\omega^{(B)}$, and computes an RIE encoding:

$$q \leftarrow \text{RIE}_{\omega^{(A)}, \omega^{(B)}}([p^{(A)}]_1, z).$$

4. User P_A secret-shares q as $[q]_0, [q]_1$.
5. User P_A sends $[q]_0, \omega^{(A)}$ and $\omega^{(B)}$ to the cloud.
6. The cloud C computes another RIE encoding and masks it with $[q]_0$:

$$t \leftarrow \text{RIE}_{\omega^{(A)}, \omega^{(B)}}([p^{(A)}]_0, [p^{(B)}]_0 + o) + [q]_0.$$

7. User P_A and the cloud C send $[q]_1$ and t , respectively, to user P_B .
8. For each $b_i \in B$:
 - (a) User P_B adds b_i to the intersection $A \cap B$ if b_i is a root of the polynomial $[q]_1 + t$.

Correctness Proof We prove here that at the end of the protocol, the querying user P_B indeed learns the intersection.

Proof. In the final step (Step 7), the querying user P_B recovers the polynomial:

$$[q]_1 + t = \text{RIE}([p^{(A)}]_0 + [p^{(A)}]_1, [p^{(B)}]_0 + o + z) = \text{RIE}(p^{(A)}, p^{(B)}),$$

due to the linearity of RIE. Therefore, the roots of this polynomial correspond to the intersection $A \cap B$, proving the correctness of the protocol. $\square$

Security Analysis We identify several security weaknesses in the protocol.

First, the distribution of the shares in Step 2 is insecure. Anyone eavesdropping on the public channel can intercept both o and z , thereby recovering the sensitive $[p^{(A)}]$, which should remain private to user P_B only.

Second, in Step 5, the sensitive random polynomials $\omega^{(A)}$ and $\omega^{(B)}$ are transmitted over the public channel. User P_B can intercept them, violating the security requirement that $\omega^{(A)}$ and $\omega^{(B)}$ must remain secret. If user P_B obtains these random polynomials, the RIE encoding $\text{RIE}(p^{(A)}, p^{(B)})$ could reveal more information than the set intersection, thereby breaking the security assumption.

Third, transmitting the share $[q]_0$ to the cloud in Step 5 and subsequently transmitting the other share $[q]_1$ to B in Step 7 is jointly insecure. User P_B can combine $[q]_0$ and $[q]_1$ to derive q . Moreover, B also receives another RIE encoding t in Step 7, which uses the same randomness $\omega^{(A)}$ and $\omega^{(B)}$. Since the derived RIE encoding q also uses the identical randomness, this violates the security requirements of RIE.

Fortunately, these weaknesses can be mitigated by transmitting these values through encrypted channels. The computing parties can initiate a key encapsulation mechanism or a key exchange protocol to securely establish a shared symmetric key. Subsequently, this key can be used to encrypt the transmitted values, ensuring that only encrypted values are sent through the public and insecure channel.

Performance To analyze the protocols computational and communication complexity, we assume that the size of each party's set is k , i.e., $|A_i| = k$, and there are N delegated parties $\{P_1, \dots, P_N\}$ and 1 querying parties.

It is easy to identify every delegated party needs to perform one RIE encoding shown in step 3, which is dominated by two polynomial multiplications. Therefore, the total computational cost is $\Omega(Nk^2)$.

As for the communication overhead, the bottleneck occurs in step 5 and step 7 where two shares $[q]_0, [q]_1$ of the RIE encoding are sent to the user P_B and thus the communication complexity is $\Omega(Nk)$.

Finally, we highlight a critical practical concern regarding deploying this PSI protocol in real-world applications. In Step 2, user P_B needs to broadcast their shares to every delegated user P_A . This effectively means that user P_B is delegating their data to each delegated user, with the delegated users themselves acting as cloud endpoints. However, this approach may not be practical as the number of delegated users increases, because the communication channels established between individual users are often less stable than the end-to-end communication between users and a centralized cloud.

Moreover, it appears infeasible to revise the protocol to eliminate the large amount of frequent end-to-end communication between users while maintaining the protocol's security. To address this issue, we present a different but efficient solution that resolves this problem without increasing the asymptotic computational and communication complexity.

3 Preliminaries

3.1 Random Intersection Encoding

This section introduces an essential (secure) tool for computing set intersections—Random Intersection Encoding (RIE). Assume the presence of a trusted third party denoted as C . Then, N computing parties $\{P_1, \dots, P_N\}$ can compute the intersection using the following RIE protocol:

We argue the correctness of Protocol 1 by proving that the protocol indeed returns the intersection $\cap_i A_i$.

Proof. Let $g = \gcd(p_1, \dots, p_N)$, then $r = \left(\sum_i \omega_i \frac{p_i}{g}\right) g$. The roots of r have two sources:

- Roots from the intersection polynomial g .
- Roots from the random polynomial $\sum_i \omega_i \frac{p_i}{g}$.

The formats of these roots are easily distinguishable:

- Roots of the random polynomial are highly unlikely (with negligible probability) to have the form $a||h(a)$.
- Roots of the intersection polynomial g have the form $a||h(a)$, which constitute the intersection $\cap_i A_i$.

Thus, the protocol is correct. $\square$

3.1.1 Security Proof

To prove the protocol's security, we show that P_i learns nothing beyond the intersection information. Equivalently, this requires proving that the output r of RIE does not reveal information beyond $\cap_i A_i$.

Randomized Intersection Encoding protocol RIE protocol	
Input	A trusted coordinator C ; N computing parties $\{P_1, \dots, P_N\}$, each holding a private set A_i .
Output	Each party learns the intersection $\bigcap_i A_i$.
1	Each party P_i re-encodes its set elements by appending a hash value: $a \leftarrow a \parallel h(a), \quad \forall a \in A_i$
2	Each P_i encodes its private set as a polynomial: $p_i(x) = \prod_{a \in A_i} (x - a)$
3	Party P_i and coordinator C agree on an encryption key. P_i sends p_i securely to C .
4	Coordinator C samples random polynomials $\omega_1, \dots, \omega_N \in \mathbb{Z}_q[x]$ and computes: $r(x) = \sum_{i=1}^N \omega_i(x) \cdot p_i(x)$
5	Coordinator C sends the resulting polynomial $r(x)$ to each party P_i via an encrypted channel.
6	Each party P_i checks which of its elements are roots of $r(x)$ to recover the intersection $\bigcap_i A_i$.

Figure 1: Randomized Intersection Encoding (RIE) protocol

Randomization Lemma Let $p_1, p_2 \in \mathbb{Z}_q^{\leq \alpha}[x]$ be coprime polynomials, and $\omega_1, \omega_2 \sim \mathbb{Z}_q^{\leq \beta}[x]$, where $\alpha \leq \beta$. Then, $\sum_{i=1}^2 p_i \omega_i$ is a random polynomial over $\mathbb{Z}_q^{\leq \alpha+\beta}$.

Proof. Consider solutions (ω_1, ω_2) to the equation $\sum_{i=1}^2 p_i \omega_i = r \in \mathbb{Z}_q^{\leq \alpha+\beta}[x]$. Assume there is a particular solution (ω_1^*, ω_2^*) . Then, the general solution can be expressed as:

$$\omega_1 = \omega_1^* - k \cdot p_2, \quad \omega_2 = \omega_2^* + k \cdot p_1, \quad k \in \mathbb{Z}_q^{\leq \beta-\alpha}[x].$$

The number of solutions equals the number of k , which is $q^{\beta-\alpha+1}$. As there are $q^{2\beta+2}$ possible combinations of (ω_1, ω_2) , we conclude that r can take any value in $\mathbb{Z}_q^{\leq \alpha+\beta}[x]$, completing the proof. $\square$

Security Proof From the randomization lemma, if $\omega_i \sim \mathbb{Z}_q^{\leq \beta}[x]$ and $\frac{p_i}{g} \sim \mathbb{Z}_q^{\leq \alpha}[x]$, then $\sum_{i=1}^N \omega_i \frac{p_i}{g}$ is a random polynomial over $\mathbb{Z}_q^{\leq \alpha+\beta}[x]$.

Finally, since $\sum_{i=1}^N \omega_i p_i \sim \mathbb{Z}_q^{\leq \alpha+\beta}[x]$, it follows that $(\sum_{i=1}^N \omega_i p_i)$ and r are indistinguishable. Thus, the protocol is secure.

3.2 Delegated Private Set Intersection

Let the query user be A_{N+1} , and adjust the RIE protocol step 5 as "only send r to A_{N+1} ". In this case, delegated private set intersection can be directly reduced to Random Intersection Encoding (RIE). The only remaining issue is that RIE assumes that C is a trusted third party that knows the private set polynomial p_i of each A_i . To remove

the trusted third-party assumption, a direct idea is to use homomorphic encryption to construct a strengthened version of RIE, which is the solution proposed. Here, we rewrite its simplified version in Protocol 2.

Delegated Multi-party PSI from Homomorphic Encryption DPSI-HE protocol	
Input	An untrusted cloud C ; N delegating parties $\{P_1, \dots, P_N\}$; one enquiring party P_{N+1} .
Output	The enquiring party P_{N+1} learns the intersection $\bigcap_{i=1}^{N+1} A_i$.
1	Each party P_i re-encodes its set elements by appending a hash value: $a \leftarrow a \parallel h(a), \quad \forall a \in A_i$
2	Each party P_i encodes its private set as a polynomial: $p_i(x) = \prod_{a \in A_i} (x - a)$
3	Parties $\{P_i\}$ and cloud C agree on a threshold homomorphic encryption scheme: each P_i holds a private key share sk_i , and C holds the public key pk .
4	Each P_i encrypts its polynomial and sends it to the cloud: $Enc_{pk}(p_i) \rightarrow C$
5	The cloud samples random polynomials $\omega_1, \dots, \omega_N \in \mathbb{Z}_q[x]$, and homomorphically computes: $Enc(r) = \sum_{i=1}^N \omega_i \cdot Enc_{pk}(p_i)$
6	The cloud sends the encrypted result polynomial $Enc(r)$ to all P_i .
7	All P_i engage in threshold decryption to recover r , ensuring only P_{N+1} learns the result.
8	Party P_{N+1} determines the intersection $\bigcap_{i=1}^{N+1} A_i$ by checking which of its elements are roots of $r(x)$.

Figure 2: Delegated Multi-party Private Set Intersection Protocol from Homomorphic Encryption

3.2.1 Brief Security Analysis

Through homomorphic encryption, P_i can securely delegate its private set to C in step 4. Additionally, C computes r in step 5 without learning p_i . Thus, the privacy of each P_i 's private set is preserved during the computation.

3.3 Multiplication Triples

In (linear) secret sharing schemes, securely performing multiplication is complex. The problem is formally described as follows:

A trusted third party (Trusted Dealer) secret-shares $x \in \mathbb{F}_p$ and $y \in \mathbb{F}_p$ as:

$$[x] = (x_1, x_2), \quad [y] = (y_1, y_2).$$

The trusted third party securely sends the shares to computing parties P_1 and P_2 via secure channels:

$$P_1 \leftarrow (x_1, y_1), \quad P_2 \leftarrow (x_2, y_2).$$

The goal is for P_1 and P_2 to compute a secret share of xy without revealing x and y .

3.3.1 Beaver's Triples

A common approach is to use Beaver's triples, i.e., pre-generated random values (a, b, c) such that $c = a \times b$. The specific steps are as follows:

Preprocessing Phase

1. The trusted third party generates a random Beaver triple (a, b, c) and creates its secret shares $([a], [b], [c])$.
2. The trusted third party securely sends:

$$P_1 \leftarrow (a_1, b_1, c_1), \quad P_2 \leftarrow (a_2, b_2, c_2).$$

Online Computation Phase

1. P_1 computes:

$$[x - a]_1 \leftarrow x_1 - a_1, \quad [y - b]_1 \leftarrow y_1 - b_1.$$

2. P_2 computes:

$$[x - a]_2 \leftarrow x_2 - a_2, \quad [y - b]_2 \leftarrow y_2 - b_2.$$

3. P_1 and P_2 collaboratively reconstruct:

$$e = x - a, \quad d = y - b.$$

4. P_1 computes its share of xy :

$$[xy]_1 \leftarrow c_1 + e \cdot b_1 + d \cdot a_1.$$

5. P_2 computes its share of xy :

$$[xy]_2 \leftarrow c_2 + e \cdot b_2 + d \cdot a_2 + e \cdot d.$$

3.3.2 Correctness of the Protocol

The correctness of the protocol is demonstrated as follows:

$$[xy]_1 + [xy]_2 = c + eb + da + ed = ab + (x - a)b + (y - b)a + (x - a)(y - b) = xy.$$

3.3.3 Security of the Protocol

- **Preprocessing Phase:** Since a trusted third party is involved, the secret shares received by P_1 and P_2 are uniformly distributed and indistinguishable from random values.
- **Online Computation Phase:** Both P_1 and P_2 receive e and d . As a and b are unknown random values, e and d appear as uniformly distributed random variables to P_1 and P_2 .
- P_1 's share $[xy]_1$ includes the random value c_1 , making it completely random from P_2 's perspective. Similarly, P_2 's share $[xy]_2$ appears random to P_1 .

Thus, the protocol preserves the secrecy of the inputs while securely computing the secret-shared product.

4 Delegated PSI Framework

This section presents a new framework for delegated PSI on the basis of Random Intersection Encoding (RIE) and secret sharing. For a polynomial

$$p = \sum_{i=0}^n p[i] \cdot x^i \in \mathbb{Z}_q^{\leq n}[x],$$

a simple $(2, 2)$ secret sharing scheme is used, where:

$$[p] = ([p]_1, [p]_2) \quad \text{such that} \quad p = [p]_1 + [p]_2.$$

Secret sharing is linear, so addition in the shares (denoted as algorithm **A_add**) is straightforward:

$$[p + q] = ([p]_1 + [q]_1, [p]_2 + [q]_2).$$

The main challenge is performing multiplication in the shares, which requires designing an algorithm **A_mul** such that:

$$P_1 \text{ holds } [p]_1, [q]_1, \quad P_2 \text{ holds } [p]_2, [q]_2, \quad \text{and computes } [p \cdot q]_1, [p \cdot q]_2.$$

4.1 Polynomial multiplication in the shares

To address this, Beaver's Triples are introduced for securely multiplying shares over $\mathbb{Z}_q$. For any $a \in \mathbb{Z}_q$, $b \in \mathbb{Z}_q$, Beaver's Triples are defined as:

$$[a \cdot b]_1, [a \cdot b]_2 \leftarrow \mathbf{A_Beaver}([a], [b]).$$

Polynomial multiplication $p \cdot q$ can be rewritten as:

$$p \cdot q = \sum_{i=0}^{2n} \sum_{j=0}^i (p[j] \cdot q[i-j]) x^i.$$

where any coefficient of the polynomials p and q with index i beyond $[0, n]$ is considered zero. Using this property, the algorithm **A_mul** is constructed as follows:

1. P_i receives its shares, including:

$$([p[0]]_i, \dots, [p[n]]_i), \quad ([q[0]]_i, \dots, [q[n]]_i).$$

2. P_i invokes **A_Beaver** to compute:

$$[p[j] \cdot q[k - j]]_i \quad \text{for all } 0 \leq k \leq 2n, 0 \leq j \leq k.$$

3. P_i calculates the sums:

$$[p \cdot q[k]]_i \leftarrow \sum_{j=0}^k [p[j] \cdot q[k - j]]_i \quad \text{for all } k.$$

4.2 Distributive Generation of Beaver's Triples

It is important to note that the method shown in the aforementioned section for generating multiplication triples assumes the existence of a trusted third party during the preprocessing phase. However, in real-world scenarios, such an idealized trusted third party often does not exist. Therefore, a critical requirement is to design a solution that allows computing parties to efficiently generate multiplication triples even in a setting where they do not trust each other. To address this challenge, we propose a construction of Beaver's triples generation protocol between the user P_i and the cloud server C under the semi-honest security model using Fully Homomorphic Encryption (FHE). The detailed algorithm for generating the random triples $([a], [b], [ab])$ is described as follows:

1. The party P_i generates a private key:

$$sk \leftarrow \mathcal{FHE}.KeyGen().$$

2. P_i randomly generates secret shares a_1, b_1 sampled from $\mathbb{F}_q$:

$$a_1, b_1 \xleftarrow{\$} \mathbb{F}_q.$$

3. C randomly generates secret shares a_2, b_2 sampled from $\mathbb{F}_q$:

$$a_2, b_2 \xleftarrow{\$} \mathbb{F}_q.$$

4. P_i encrypts his secret shares:

$$\text{Enc}_{sk}(a_1) \leftarrow \mathcal{FHE}.Encrypt(sk, a_1), \quad \text{Enc}_{sk}(b_1) \leftarrow \mathcal{FHE}.Encrypt(sk, b_1),$$

and sends these encrypted values to C through a secret channel between P_i and C .

5. Party C performs homomorphic additions to compute:

$$\text{Enc}_{sk}(a) \leftarrow a_2 + \text{Enc}_{sk}(a_1), \quad \text{Enc}_{sk}(b) \leftarrow b_2 + \text{Enc}_{sk}(b_1).$$

6. Party C performs a homomorphic multiplication to compute the encrypted product:

$$\text{Enc}_{sk}(ab) \leftarrow \text{Enc}_{sk}(a) \cdot \text{Enc}_{sk}(b).$$

7. Party C randomly generates a secret share:

$$c_2 \xleftarrow{\$} \mathbb{F}_q.$$

8. Party C computes another secret share of the product:

$$\text{Enc}_{sk}(c_1) \leftarrow \text{Enc}_{sk}(ab) - c_2.$$

and sends the computed value to P_i through a secret channel between P_i and C .

9. Party P_i decrypt $\text{Enc}_{sk}(c_1)$:

$$c_1 \leftarrow \mathcal{FHE}.Decrypt(sk, \text{Enc}_{sk}(c_1)).$$

Remarks The user P_i first generates a private key, then encrypts his secret shares a_1 and b_1 , and sends the encrypted values to C . Party C computes the homomorphic addition and multiplication to obtain the encrypted ab . Subsequently, the two parties sequentially retrieve the secret shares of ab , i.e., $[ab] = (c_1, c_2)$ ensuring both privacy and correctness.

4.3 Key Exchange

To ensure the functionality of our PSI framework, all computing parties, including $A_1, \dots, A_{N+1}$, must share a common secret key s^* without the cloud server knowing the key. To adhere to the definition of delegated PSI, all communication must go through the cloud, avoiding direct communication between users. We propose the following simple yet efficient key exchange protocol based on any semantically secure public key encryption scheme $\mathcal{PKE}$:

1. Each P_i invokes the key generation algorithm to generate its key pair:

$$sk_i, pk_i \leftarrow \mathcal{PKE}.\text{KeyGen}().$$

2. Each P_i broadcasts its public key pk_i via the cloud to all other parties $P_j, j \neq i$.
3. One randomly selected party among $\{P_i\}_i$ generates a common symmetric secret key s^* and encrypts it for each $P_j, j \neq i$:

$$c_j \leftarrow \mathcal{PKE}.\text{Encrypt}(pk_j, s^*),$$

and broadcasts c_j via the cloud to P_j .

4. Each P_j decrypts c_j using its private key sk_j to obtain the common symmetric secret key s^* .

4.4 New Framework

With the capability to perform addition and multiplication on secret shares, we present a complete delegated multi-party PSI framework in Protocol 3. We assume that the common secret key s^* and the Beaver's triples have been pre-generated prior to the initiation of the PSI protocol.

Remarks The correctness of the algorithm can be proved using the analogous argument we used to prove Protocol 2. We focus here to argue that the proposed framework is secure against semi-honest adversaries.

Proof. The security of the protocol is proved using the simulation paradigm. The ideal model is described as follows:

1. Each computing party P_i holds a private set A_i and constructs the corresponding set polynomial p_i .
2. The cloud server C holds private random polynomials ω_i for all i .
3. Through some "supernatural" operation (oracle machine access), P_{N+1} directly obtains:

$$r = \sum_i \omega_i p_i,$$

and computes the intersection:

$$\cap_i A_i.$$

Delegated Multi-party PSI Using Secret Sharing DPSI-SS protocol	
Input	An untrusted cloud server C ; N delegating parties $\{P_1, \dots, P_N\}$; one querying party P_{N+1} .
Output	The querying party P_{N+1} learns the intersection $\bigcap_{i=1}^{N+1} P_i$.
1	Each party P_i re-encodes each set element $a \in A_i$ as: $a \leftarrow a \parallel h(a)$
2	Each party P_i encodes its private set as a polynomial: $p_i(x) = \prod_{a \in A_i} (x - a)$
3	Each P_i creates a secret sharing of p_i , denoted as $[p_i] = ([p_i]_0, [p_i]_1)$. It retains $[p_i]_1$ and sends $[p_i]_0$ to the cloud server C .
4	Each P_i samples a random polynomial ω_i , creates shares $[\omega_i] = ([\omega_i]_0, [\omega_i]_1)$, retains $[\omega_i]_1$, and sends $[\omega_i]_0$ to C .
5	The cloud C homomorphically computes: $\left[\sum_{i=1}^N \omega_i p_i \right]_0 = \sum_{i=1}^N [\omega_i]_0 \cdot [p_i]_0$
	using primitives A_add and A_mul .
6	Each P_i locally computes: $[\omega_i p_i]_1 = [\omega_i]_1 \cdot [p_i]_1$
	using A_mul .
7	The cloud sends $\left[\sum_{i=1}^N \omega_i p_i \right]_0$ to P_{N+1} via a secure channel.
8	Each P_i encrypts $[\omega_i p_i]_1$ using a common secret key s^* , and sends it to C , who forwards it to P_{N+1} .
9	The querying party P_{N+1} decrypts $[\omega_i p_i]_1$ for all i , and reconstructs: $r(x) = \sum_{i=1}^N \omega_i(x) \cdot p_i(x)$
	Then it determines the intersection $\bigcap_i P_i$ by testing which of its elements are roots of $r(x)$.

Figure 3: Delegated Multi-party Private Set Intersection Protocol Using Secret Sharing

To show security, simulators S_i , S_c , and S_{N+1} are constructed to mimic the views of P_i , C , and P_{N+1} , respectively, in the real world.

Simulator S_i simulates P_i 's perspective in the real-world protocol (Protocol 3):

1. Generate random polynomials:

$$r_i \leftarrow \mathbb{Z}_q^{\leq k}[x] \quad \text{for all } i,$$

to simulate Step 3 in the real model.

2. Generate random polynomials:

$$s_i \leftarrow \mathbb{Z}_q^{\leq k}[x] \quad \text{for all } i,$$

to simulate Step 5 in the real model.

3. Generate two additional random polynomials to simulate Steps 8 and 9 in the real model.

Simulator S_c simulates C 's perspective in the real-world protocol:

1. Generate random polynomials:

$$r_i \leftarrow \mathbb{Z}_q^{\leq k}[x] \quad \text{for all } i,$$

to simulate Step 3 from C 's perspective in the real model.

2. Generate a random polynomial:

$$s \leftarrow \mathbb{Z}_q^{\leq k}[x],$$

to simulate Step 9 from C 's perspective in the real model.

Simulator S_{N+1} simulates P_{N+1} 's perspective in the real-world protocol:

1. Generate random polynomials:

$$r_i \leftarrow \mathbb{Z}_q^{\leq k}[x] \quad \text{for all } i,$$

to simulate Step 3 from P_{N+1} 's perspective.

2. Generate random polynomials:

$$s_i \leftarrow \mathbb{Z}_q^{\leq k}[x] \quad \text{for all } i,$$

to simulate Step 5 from P_{N+1} 's perspective.

3. Extract the real-world value of: $r = \sum_i \omega_i p_i$, and construct secret shares: $[r]_0, \dots, [r]_N$.
4. Encrypt $[r]_0$ to simulate Step 8 as $\text{Enc}([r]_0)$.
5. Encrypt $[r]_1, \dots, [r]_N$ to simulate Step 9 as $\text{Enc}([r]_1), \dots, \text{Enc}([r]_N)$.

The views generated by simulators S_i , S_c , and S_{N+1} are indistinguishable from the real-world protocol executions. This is because the random polynomials r_i and s_i do not reveal any private information about the sets A_i or the polynomials p_i , and the encryption ensures that the cloud server C and other parties cannot access unauthorized information. Thus, the protocol securely emulates the ideal model. $\square$

4.5 Performance

To analyze the protocol's computational and communication complexity, we assume that the size of each party's set is k , i.e., $|A_i| = k$.

Computational Complexity The computational bottleneck of the protocol occurs in Step 6, which involves multiple invocations of $\mathbf{A_mul}$. Specifically, Step 6 performs $\Omega(N)$ polynomial multiplications in the share state. Each polynomial multiplication requires $\Omega(k^2)$ $\mathbf{A_mul}$ invocations, and each $\mathbf{A_mul}$ invocation completes in constant time. Consequently, the total computational complexity of the protocol is:

$$\Omega(Nk^2).$$

Communication Complexity The polynomial p_i representing the set A_i has a length of $k \log q$ bits. The communication bottleneck arises in Steps 3 and 4, where $\Omega(N)$ polynomials are transmitted in total. Thus, the total communication complexity is:

$$\Omega(Nk \log q).$$

5 Optimization Strategies

We propose the following techniques to reduce the computational complexity of the protocol.

5.1 Binning Technique

We introduce another set representation method: the hash table (HT) on top of the set polynomial representation.

The hash table of a set S is a two-dimensional linked list (or equivalently, a matrix) denoted as $\text{HT}[\cdot]$, which satisfies:

$$\forall s \in S \Leftrightarrow s \in \text{HT}[H(s)],$$

where $H(\cdot) : \{0, 1\}^* \rightarrow \{0, \dots, h-1\}$ is a hash function. The algorithmic process (denoted as $\mathcal{A}_{ht}$) for constructing a hash table for a set S proceeds as follows:

1. For every $s \in S$:
 - (a) Compute the address $H(s)$.
 - (b) Access the corresponding position $H(s)$ in the hash table, $\text{HT}[H(s)]$.
 - (c) Insert the element s into $\text{HT}[H(s)]$.
2. return the hash table HT

Remark Each position $j \in \{0, 1, \dots, h-1\}$ in the hash table may contain multiple elements from the set S . Thus, $\text{HT}[j]$ itself forms a small subset comprising elements that share the same hash address. In the literature, these subsets $\text{HT}[j]$ are commonly referred to as "bins."

According to the balls-into-bins principle, when the row number of the hash table (denoted as h) is asymptotically as large as set size $|S|$ (denoted as k), such that $h = \Omega(k)$, each bin will contain at most $\Omega(\log k)$ elements with high probability. For security reason, the number of elements in each row should be identical to prevent information leakage on the dataset distribution. This can be achieved by inserting 'dummy' items (denoted as s_{dummy}) to make the length of each row identical. In other terms, the size of the hash table should be $\Omega(k \log k)$. Moreover, with this technique, the computational complexity in Step 6 of Algorithm 3 is reduced to $\Omega(Nk(\log k)^2)$.

Numerical Analysis Nevertheless, in practical implementations, it is essential to compute the exact value of the maximum load, defined as the maximum number of set elements in a single row of the hash table, to optimize storage overhead. Asymptotic analysis alone does not provide sufficient precision for this purpose. To address this, we introduce a numerical computation method based on the Chernoff bounds.

Chernoff Bounds are a powerful tool for estimating the probability that a random variable deviates significantly from its expected value. They are particularly useful for analyzing large-scale independent random events. The upper tail of the Chernoff Bound

is specifically used to study cases where a random variable exceeds its expectation significantly, which is precisely our concern when analyzing the upper deviation of a distribution.

Consider $X_1, X_2, \dots, X_k$ as k independent Bernoulli random variables, where each X_i takes values in $\{0, 1\}$. Define $X = \sum_{i=1}^k X_i$ as their sum, with an expected value $\mu = \mathbb{E}[X]$. The upper tail Chernoff Bound provides an upper limit on the probability that X deviates above its expectation:

$$\Pr(X \geq (1 + \delta)\mu) \leq \exp\left(-\frac{\delta^2\mu}{2 + \delta}\right)$$

where $\delta > 0$ represents the deviation parameter, meaning that $(1 + \delta)\mu$ denotes a deviation of $\delta\mu$ beyond the expectation μ . It is evident that as δ increases, the probability of exceeding the expected value decreases exponentially.

Returning to the maxload problem, let $\mu = \frac{k}{h}$. The upper tail Chernoff Bound formula provides an estimate for the maximum load of a single bin. However, in the maxload problem, we need to estimate the probability that *none* of the h rows of the hash table exceeds the maximum load. To achieve this, we introduce the Union Bound.

For a set of events $E_1, E_2, \dots, E_h$, the probability of their union satisfies the following inequality:

$$\Pr\left(\bigcup_{i=1}^h E_i\right) \leq \sum_{i=1}^h \Pr(E_i)$$

The Union Bound states that the probability of at least one of multiple events occurring does not exceed the sum of their individual probabilities. In other words, it provides an upper bound for the probability of a union of events.

Clearly, if we define E_i as the event that "the number of set elements in the i -th bin exceeds the maximum load," then $\Pr\left(\bigcup_{i=1}^h E_i\right)$ represents the probability that *at least one* of the h hash table rows exceeds the maximum load. Applying the Union Bound, we obtain:

$$\Pr\left(\bigcup_{i=1}^h E_i\right) \leq h \cdot \exp\left(-\frac{\delta^2 \cdot k}{h(2 + \delta)}\right)$$

Based on the union bound formula, we construct an algorithm to provide a numerical estimation of the maximum load and the number of hash table rows as follows:

Input: Total number of set elements N_{elts} , bias parameter δ , target probability P_{target}

Output: Maximum load per row L_{max} , final number of hash table rows N_{rows} , computed probability P_{computed}

- 1 Set initial number of hash table rows $N_{\text{rows}} \leftarrow N_{\text{elts}}$;
- 2 Compute average load per bin $L_{\text{avg}} \leftarrow \frac{N_{\text{elts}}}{N_{\text{rows}}}$;
- 3 Compute initial probability bound:

$$P_{\text{computed}} \leftarrow N_{\text{rows}} \cdot \exp\left(-\frac{\delta^2 \cdot L_{\text{avg}}}{2 + \delta}\right)$$
- 4 **while** $P_{\text{computed}} > P_{\text{target}}$ **do**
 - 5 Decrease the number of hash table rows $N_{\text{rows}} \leftarrow N_{\text{rows}} - 0.01 \cdot L_{\text{avg}}$
 - 6 Recompute the average load $L_{\text{avg}} \leftarrow \frac{N_{\text{elts}}}{N_{\text{rows}}}$
 - 7 Recompute the probability bound:

$$P_{\text{computed}} \leftarrow N_{\text{rows}} \cdot \exp\left(-\frac{\delta^2 \cdot L_{\text{avg}}}{2 + \delta}\right)$$
- 8 Compute the maximum load $L_{\text{max}} \leftarrow \lfloor (1 + \delta) \cdot L_{\text{avg}} \rfloor$
- 9 Set final number of hash table rows $N_{\text{rows}} \leftarrow \lfloor N_{\text{rows}} \rfloor$
- 10 **return** $(L_{\text{max}}, N_{\text{rows}}, P_{\text{computed}})$

Algorithm 1: Computation of Maximum Load Per Row and Row Allocation

Finally, we provide several concrete parameter sets used in our actual PSI implementation, as shown in Table 1. In these configurations, we set the probability of "exceeding the maximum load" to a negligible value, i.e.,

$$P_{\text{target}} \stackrel{\text{def}}{=} \Pr\left(\bigcup_{i=1}^h T_i\right) \leq 2^{-40},$$

and set the deviation parameter δ to 1.5, meaning that the maximum load is 2.5 times the average load $\frac{k}{h}$.

Table 1: Concrete hash table parameters with estimated maximum loads.

Set Size k	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
Row Number h	21	81	314	1209	4658	17964
Deviation Parameter δ	1.5	1.5	1.5	1.5	1.5	1.5
Maximum Load	120	124	130	135	140	145

5.2 Fast Fourier Transform

When implementing the proposed protocols, the author encounters the following bottleneck problem: given two integer-coefficient polynomials

$$a = \sum_{i=0}^{n-1} a_i x^i, \quad b = \sum_{i=0}^{n-1} b_i x^i \in \mathbb{F}_q[x],$$

how can one efficiently compute the product $a \cdot b$?

The classical method relies on the convolution formula

$$a \cdot b = \sum_{i=0}^{2n-2} \left(\sum_{j=0}^i a_j b_{i-j} \right) x^i \mod q,$$

which performs polynomial multiplication in $\Omega(n^2)$ time.

We explore an alternative approach to accelerate polynomial multiplication: representing the polynomials a and b by their evaluations at a set of specially chosen points $\{w_k\}_{k=0}^{2n-1}$, i.e., using their values at those points (point-value representation), denoted as $\{a(w_k)\}$ and $\{b(w_k)\}$ respectively. Under this representation, the product $a \cdot b$ corresponds to pointwise multiplication at each evaluation point:

$$\{a(w_k) \cdot b(w_k)\}.$$

Note that this pointwise multiplication step only requires $\Omega(n)$ time. Therefore, if both the evaluation (from coefficient form to point-value form) and interpolation (from point-value form back to coefficient form) can be performed in $\Omega(n \log n)$ time, the overall multiplication can be significantly accelerated. This observation motivates the use of the Fast Fourier Transform (FFT).

5.2.1 Intuition

Fast Fourier Transform In many scenarios, we are not primarily concerned with the signal's value at each time point, but rather with its frequency components. The Discrete Fourier Transform (DFT) serves precisely this purpose by projecting a signal onto a basis of complex exponentials for frequency spectrum analysis.

Formally, the DFT transforms a sequence of length n , denoted by $(x_0, x_1, \dots, x_{n-1})$, into a new sequence $(X_0, X_1, \dots, X_{n-1})$ using the formula:

$$X_k = \sum_{i=0}^{n-1} x_i \cdot e^{-2\pi j i k / n}, \quad \text{for } k = 0, 1, \dots, n-1,$$

where j is the imaginary unit.

It is easy to see that direct computation of the DFT requires $\Omega(n^2)$ time. In 1965, Cooley and Tukey exploited the symmetry properties of the so-called twiddle factor $\omega := e^{-2\pi j / n}$ and proposed the Fast Fourier Transform (FFT), which reduces the time complexity to $\Omega(n \log n)$.

Frequency-Domain Representation of Polynomials From the perspective of the Fast Fourier Transform (FFT), one can interpret FFT as evaluating a polynomial

$$a(x) = \sum_{i=0}^{n-1} a_i x^i$$

at n distinct complex roots of unity, known as "twiddle factors", defined as

$$\omega_k \stackrel{\text{def}}{=} e^{-2\pi j k / n}, \quad \text{for } k = 0, 1, \dots, n-1.$$

That is, the FFT computes

$$a(\omega_k) = \sum_{i=0}^{n-1} a_i \omega_k^i.$$

However, note that the product $a(x) \cdot b(x)$ has degree at most $2n - 2$, and hence requires at least $2n - 1$ evaluation points to be uniquely determined. Therefore, using only n evaluation points is insufficient to reconstruct the product.

To resolve this, we define $\text{fft}_{2n}(a)$ as the evaluation of $a(x) = \sum_{i=0}^{n-1} a_i x^i$ at $2n$ complex roots of unity:

$$\omega_k^{(2n)} \stackrel{\text{def}}{=} e^{-2\pi j k / (2n)}, \quad \text{for } k = 0, 1, \dots, 2n - 1,$$

so that

$$\text{fft}_{2n}(a) \stackrel{\text{def}}{=} \left\{ a(\omega_k^{(2n)}) \right\}_{k=0}^{2n-1}.$$

In this case, given $\text{fft}_{2n}(a \cdot b)$, one can uniquely recover the product polynomial ab .

Proof (by contradiction). Assume that the reconstruction is not unique, i.e., suppose two different polynomials $a(x)$ and $a'(x)$ yield the same evaluation vector at the $2n$ roots of unity. Let $h(x) = a(x) - a'(x)$. Then $h(\omega_k^{(2n)}) = 0$ for all $k = 0, 1, \dots, 2n - 1$. This implies that $h(x)$ has $2n$ distinct roots, but $\deg(h) \leq 2n - 1$, which contradicts the Fundamental Theorem of Algebra. Therefore, the reconstruction must be unique. $\square$

It follows that

$$\text{fft}_{2n}(a \cdot b) = \text{fft}_{2n}(a) \odot \text{fft}_{2n}(b),$$

where $\odot$ denotes element-wise multiplication. Based on this observation, we present the FFT-based polynomial multiplication algorithm as follows:

1. Compute the evaluation vector: $A \leftarrow \text{fft}_{2n}(a)$
2. Compute the evaluation vector: $B \leftarrow \text{fft}_{2n}(b)$
3. Perform element-wise multiplication: $C \leftarrow A \odot B$
4. Compute the inverse FFT to recover the result: $ab \leftarrow \text{fft}_{2n}^{-1}(C)$
5. Return $ab \bmod q$

This algorithm performs two forward FFTs and one inverse FFT. Each FFT or inverse FFT takes $\Omega(n \log n)$ time, so the entire multiplication can be completed in $\Omega(n \log n)$ time.

5.2.2 Precision Control

Since computers represent real numbers using floating-point approximations, exact values cannot be obtained in Steps 1 and 2 of the FFT-based algorithm. This introduces a core challenge in implementing the FFT-based approach: *How can we choose appropriate FFT parameters to ensure that the algorithm is error-free, i.e., it always computes the correct product polynomial despite numerical approximations?*

To address this issue, we propose a *precision loss estimation* framework that analyzes the numerical errors introduced by floating-point arithmetic during FFT computations.

First, consider the error introduced by multiplying two real numbers $a, b \in \mathbb{R}$. Let their floating-point approximations be denoted by $\tilde{a}, \tilde{b}$, satisfying:

$$\tilde{a} \leq a(1 + \epsilon_0), \quad \tilde{b} \leq b(1 + \epsilon_0),$$

where ϵ_0 denotes the machine (relative) precision. Then,

$$\tilde{a} \cdot \tilde{b} = ab(1 + 2\epsilon_0 + \epsilon_0^2),$$

so the relative error in computing $a \cdot b$ is approximately $2\epsilon_0 + o(\epsilon_0) \approx 2\epsilon_0$.

Next, consider the multiplication of two complex numbers $a, b \in \mathbb{C}$, written as $a = a_0 + ia_1$ and $b = b_0 + ib_1$. Their product is given by:

$$a \cdot b = (a_0b_0 - a_1b_1) + i(a_0b_1 + a_1b_0).$$

The structure of the real and imaginary parts is symmetric: each consists of two real multiplications and one real addition. Since multiplication dominates in error contribution, and addition errors can be neglected, the relative error for both the real and imaginary parts remains approximately $2\epsilon_0$.

A single n -point FFT consists of $\log n$ layers of butterfly operations, and each butterfly performs one multiplication and two additions. Thus, the dominant source of numerical error arises from the $\log n$ multiplications.

Furthermore, each multiplication in the butterfly structure involves a data input x and a twiddle factor ω_n^k . Assuming x has a relative error of ϵ_x , and the real (or imaginary) part of ω_n^k has relative error ϵ_0 , the total relative error introduced by one butterfly operation is roughly:

$$\epsilon_x + \epsilon_0.$$

As a result, the relative error of a single FFT can be estimated as:

$$\log n \cdot \epsilon_0. \tag{1}$$

Therefore, when performing a forward FFT followed by an inverse FFT on an integer-coefficient polynomial $a(x) \in \mathbb{Z}[x]$, the total relative error can be estimated as:

$$2 \log n \cdot \epsilon_0.$$

Using double-precision floating-point numbers (with $\epsilon_0 \approx 2^{-53}$), we test two commonly used parameter sets. The results are summarized below:

- ($n = 2^{10}$, $\epsilon_{\text{estimate}} = 10^{-15}$, $\epsilon_{\text{real}} = 10^{-16}$)
- ($n = 2^{20}$, $\epsilon_{\text{estimate}} = 10^{-15}$, $\epsilon_{\text{real}} = 10^{-16}$)

Our numerical experiment results show that the FFT exhibits excellent numerical stability, with actual errors approaching machine precision.

When using the FFT-based polynomial multiplication algorithm, computational errors arise from three main sources:

1. The forward FFT on the input polynomials: $a \leftarrow \text{fft}_{2n}(a)$, $b \leftarrow \text{fft}_{2n}(b)$
2. Pointwise multiplication of the evaluation vectors: $a \odot b$
3. The inverse FFT to recover the output polynomial: $ab \leftarrow \text{fft}_{2n}^{-1}(a \odot b)$

The error introduced in each step can be estimated as follows:

1. The forward FFT introduces an error of approximately $\log(2n) \cdot \epsilon_0$.
2. The pointwise multiplication also contributes an error of approximately $\log(2n) \cdot \epsilon_0$.
3. The inverse FFT adds another $\log(2n) \cdot \epsilon_0$.

Therefore, the total relative error in the FFT-based polynomial multiplication can be estimated as:

$$3 \log(2n) \cdot \epsilon_0. \tag{2}$$

Using double-precision floating-point numbers (i.e., $\epsilon_0 \approx 2^{-53}$), we test two commonly used parameter sets. The estimated and actual errors are summarized below:

- $(n = 2^{10}, \quad \epsilon_{\text{estimate}} = 10^{-15}, \quad \epsilon_{\text{real}} = 10^{-16})$
- $(n = 2^{15}, \quad \epsilon_{\text{estimate}} = 10^{-15}, \quad \epsilon_{\text{real}} = 10^{-16})$

These results again confirm the high numerical stability of FFT-based polynomial multiplication: the actual error remains very close to machine precision.

5.2.3 A New Method for Polynomial Multiplication

In addition to numerical precision errors, floating-point numbers also have limitations in representable range. For instance, double-precision floating-point numbers can represent values up to

$$(1 - 2^{-52}) \times 2^{1023}.$$

Moreover, the spacing between representable numbers depends on the magnitude: the smaller the value, the finer the resolution; the larger the value, the coarser the resolution. For example, in double precision:

- Near 2^0 : machine precision is approximately 2^{-52} ,
- Near 2^{20} : machine precision is approximately 2^{-32} ,
- Near 2^{100} : machine precision is approximately 2^{-48} .

We exploit the high precision of floating-point numbers in the interval $[0, 1)$ to convert arithmetic over $\mathbb{Z}_p$ into the interval $[0, 1)$. This transformation enables accurate addition and multiplication while maintaining high numerical stability. Based on this idea, we construct an FFT-based algorithm for multiplying integer-coefficient polynomials as follows:

Input: integer polynomial a and b , and modulus q

Output: $ab \bmod q$

- 1 Compute evaluation vectors: $a' \leftarrow \text{fft}_{2n}(a/q)$
- 2 Compute evaluation vectors: $b' \leftarrow \text{fft}_{2n}(b/q)$
- 3 Perform pointwise multiplication: $a' \odot b'$
- 4 Inverse FFT to recover product: $a'b' \leftarrow \text{fft}_{2n}^{-1}(a' \odot b')$
- 5 Output: $\lfloor a'b' \cdot q^2 \rfloor \bmod q$

Algorithm 2: FFT-based Integer Polynomial Multiplication

Remark 1 (Correctness Analysis). Assuming no floating-point errors for simplicity, Step 4 gives

$$a'b' = \frac{ab}{q^2},$$

so Step 5 correctly recovers $ab \bmod q$.

Remark 2 (Precision Analysis). Let ϵ_0 denote the machine precision. Then:

- Step 1 yields an approximation of a' : $(1 + \log(2n) \cdot \epsilon_0) \cdot a'$.
- Step 2 gives: $(1 + \log(2n) \cdot \epsilon_0) \cdot b'$.
- Step 3 produces: $(1 + 2\log(2n) \cdot \epsilon_0) \cdot (a' \odot b')$.
- Step 4 gives: $(1 + 3\log(2n) \cdot \epsilon_0) \cdot a'b'$.

To ensure that Step 5 correctly recovers ab , we require:

$$3 \log(2n) \cdot \epsilon_0 \cdot \|ab\|_\infty < \frac{1}{2}.$$

Since $\|ab\|_\infty < nq^2$, we obtain the bound:

$$\epsilon_0 < \frac{1}{6n \cdot \log(2n) \cdot q^2}. \quad (3)$$

Example Parameters. Assuming $\epsilon_0 \approx 2^{-53}$ (double precision), the following are typical parameter choices that satisfy the bound:

- $(n = 2^{10}, \quad q = 2^{18})$
- $(n = 2^{15}, \quad q = 2^{15})$
- $(n = 2^{20}, \quad q = 2^{13})$

5.2.4 Extension to Large Modulus

The method proposed in this paper is applicable primarily when the modulus q is relatively small (e.g., $q \approx 2^{13}$). We now discuss how to extend this method to the case where q is large.

Assume that the large modulus q can be expressed as the product of ℓ pairwise coprime smaller moduli $\{q_i\}_{i=1}^\ell$:

$$q = \prod_{i=1}^{\ell} q_i.$$

By the Chinese Remainder Theorem, we have the isomorphism:

$$\mathbb{Z}_q[x] \simeq (\mathbb{Z}_{q_1}[x], \dots, \mathbb{Z}_{q_\ell}[x]).$$

Therefore, to compute $ab \bmod q$, it suffices to apply our method independently under each small modulus q_i , obtaining:

$$x_i \stackrel{\text{def}}{=} ab \bmod q_i \quad \text{for all } i = 1, \dots, \ell,$$

and then reconstruct $ab \bmod q$ using the CRT reconstruction formula:

$$ab = \sum_{i=1}^{\ell} x_i \cdot M_i \cdot M_i^{-1} \bmod q,$$

where $M_i = q/q_i$, and M_i^{-1} satisfies $M_i \cdot M_i^{-1} \equiv 1 \pmod{q_i}$.

Finally, we consider how to compress FFT data to reduce memory usage.

For any real-valued sequence $(x_0, x_1, \dots, x_{n-1}) \in \mathbb{R}^n$, its DFT yields a complex-valued frequency spectrum $(X_0, X_1, \dots, X_{n-1}) \in \mathbb{C}^n$ that satisfies the conjugate symmetry property:

$$X_{n-k} = \overline{X_k}, \quad \text{for } k = 1, \dots, n-1.$$

This conjugate symmetry implies that only the first half of the frequency domain needs to be stored explicitly, while the second half can be reconstructed efficiently. As a result, the storage requirement of FFT-based polynomial multiplication can be reduced by approximately 50%.

Final Remark We re-calculate the computational complexity of our PSI protocol after incorporating the two optimization techniques aforementioned. Note that each party's hash table has $\Omega(k)$ rows, and each row contains $\Omega(\log k)$ set elements. Thus, each row is represented by a set polynomial of degree $\Omega(\log k)$. As discussed earlier, the overall protocol requires $\Omega(Nk)$ polynomial multiplications. Since each fft-based polynomial multiplication has a computational cost of $\Omega(\log k \cdot \log \log k)$, the total computation cost for polynomial multiplications is $\Omega(Nk \log k \log \log k)$. Therefore, we conclude that the overall complexity of the protocol is $\tilde{\Omega}(Nk)$.

5.2.5 Acceleration by parallel-computing

Here we consider introducing parallel computing techniques to accelerate polynomial multiplication. For ease of analysis, we adopt the PRAM (Parallel Random Access Machine) computational model, which assumes p processors sharing a global memory, where each processor can access any memory cell in constant time.

Internal Parallelism A single polynomial multiplication contains inherent parallelism, which we call the *internal parallel algorithm*.

Consider the polynomial multiplication

$$c(X) = a(X) \cdot b(X) \bmod q,$$

where a, b are polynomials with $\deg(a) = \deg(b) = n - 1$, and the modulus q factors as

$$q = \prod_{i=1}^{\ell} q_i.$$

The internal parallel algorithm for polynomial multiplication can be described as follows:

Input: Polynomials $a(X), b(X)$ with $\deg(a) = \deg(b) = n - 1$, modulus $q = \prod_{i=1}^{\ell} q_i$

Output: $c(X) = a(X) \cdot b(X) \bmod q$

```

1 for  $i = 1, 2, \dots, \ell$  in parallel do
2    $a_i \leftarrow a \bmod q_i$ ;
3    $b_i \leftarrow b \bmod q_i$ ;
4 for  $i = 1, 2, \dots, \ell$  in parallel do
5    $a'_i \leftarrow \text{fft}_{2n}(\frac{a_i}{q_i})$ ;
6    $b'_i \leftarrow \text{fft}_{2n}(\frac{b_i}{q_i})$ ;
7    $a'_i \odot b'_i \leftarrow$  element-wise multiplication;
8    $a'_i b'_i \leftarrow \text{fft}_{2n}^{-1}(a'_i \odot b'_i)$ ;
9    $x_i \leftarrow [a'_i b'_i \cdot q_i^2] \bmod q_i$ ;
10 for  $i = 1, 2, \dots, \ell$  in parallel do
11    $x'_i \leftarrow x_i \cdot M_i \cdot M_i^{-1} \bmod q$ ;
12 return  $c(X) = \sum_{i=1}^{\ell} x'_i \bmod q$ ;

```

Algorithm 3: Internal Parallel Polynomial Multiplication

It is clear that except for the summation in Step 4, the other steps are highly parallelizable. The computational bottleneck lies in Steps 2.1, 2.2, and 2.4, resulting in a total of 3ℓ FFT operations. If the number of processors satisfies $1 \leq p \leq \ell$, the runtime is $\frac{3\ell}{p}$ FFT operations, i.e., the overall runtime is

$$\Omega\left(\frac{\ell n \log n}{p}\right).$$

External Parallelism In addition to internal parallelism, polynomial multiplication also admits external parallelism, which we call the *external parallel algorithm*.

Suppose the computational task consists of m independent polynomial multiplications:

$$c^{(i)}(X) = a^{(i)}(X) \cdot b^{(i)}(X) \bmod q, \quad i = 1, \dots, m.$$

Distributing these m multiplications evenly among p processors yields up to a p -fold speedup.

Hybrid Parallelism Finally, one may combine both internal and external parallelism, which we call the *hybrid parallel algorithm*. Specifically, the m polynomial multiplications are first partitioned into p_e parallel subtasks, each responsible for m/p_e multiplications. Then, for each subtask, $1 \leq p_i \leq \ell$ processors are allocated to execute the internal parallel algorithm.

It follows that the hybrid parallel algorithm achieves an overall speedup of $p_e \cdot p_i$ in the ideal case.

5.3 Extended Beaver's multiplication triple

5.3.1 Problem Formulation

We previously considered in Section 4.1 the use of standard Beaver multiplication triples to perform polynomial multiplication on secret-shared inputs. However, this approach incurs a computational overhead of $\Omega(n^2)$, which can become a bottleneck for large-degree polynomials. This naturally leads to the following question: *Can we overcome the $\Omega(n^2)$ computational barrier to achieve faster polynomial multiplication in the secret-sharing setting?*

We answer this question affirmatively by generalizing the Beaver multiplication triple to the setting where $x, y \in \mathbb{Z}_p^{\leq n}[x]$, i.e., polynomials over $\mathbb{Z}_p$ of degree at most n . We further investigate how to securely and efficiently compute their product under secret sharing.

The problem is formalized as follows: A trusted dealer secret-shares the polynomials $x, y \in \mathbb{Z}_p[x]$ into additive shares $[x] = (x_1, x_2)$, $[y] = (y_1, y_2)$, and sends them over secure channels to two parties P_1 and P_2 :

$$P_1 \leftarrow (x_1, y_1), \quad P_2 \leftarrow (x_2, y_2)$$

The two parties aim to compute additive shares of the product xy , without learning anything about the original polynomials.

5.3.2 Polynomial Beaver Triples

The PSI protocol proposed in this paper operates on a more complex algebraic structure: specifically, the basic elements (e.g., each party P_i encoding its input set as a polynomial p_i) are defined over the polynomial ring $\mathbb{Z}_q[x]$, where

$$\mathbb{Z}_q[x] = \{a_0 + a_1x + \dots + a_{n-1}x^{n-1} \mid a_i \in \mathbb{Z}_q, n \geq 0\}$$

Addition and multiplication in $\mathbb{Z}_q[x]$ are defined as:

$$\begin{aligned} \sum_{i=0}^n a_i x^i + \sum_{i=0}^n b_i x^i &= \sum_{i=0}^n (a_i + b_i) x^i \\ \left(\sum_{i=0}^n a_i x^i \right) \cdot \left(\sum_{i=0}^n b_i x^i \right) &= \sum_{i=0}^{2n} \left(\sum_{j=0}^i a_j \cdot b_{i-j} \right) x^i \end{aligned}$$

To adapt to this, we introduce *polynomial Beaver triples*, i.e., preprocessed random triples $(a, b, c) \in \mathbb{Z}_q^{\leq n}[x] \times \mathbb{Z}_q^{\leq n}[x] \times \mathbb{Z}_q^{\leq 2n}[x]$ satisfying $c = ab$.

Preprocessing Phase: The trusted dealer generates a random Beaver triple (a, b, c) and secret-shares it as $([a], [b], [c])$. It sends

$$P_1 \leftarrow (a_1, b_1, c_1), \quad P_2 \leftarrow (a_2, b_2, c_2)$$

Online Computation Phase:

1. The parties compute their respective shares

$$\begin{aligned} [x - a]_1 &\leftarrow x_1 - a_1, & [y - b]_1 &\leftarrow y_1 - b_1 \\ [x - a]_2 &\leftarrow x_2 - a_2, & [y - b]_2 &\leftarrow y_2 - b_2 \end{aligned}$$

2. The parties exchange their shares and reconstruct:

$$e := x - a, \quad d := y - b$$

3. Then, each party locally computes a share of xy as:

$$\begin{aligned} [xy]_1 &\leftarrow c_1 + e \cdot b_1 + d \cdot a_1 \\ [xy]_2 &\leftarrow c_2 + e \cdot b_2 + d \cdot a_2 + e \cdot d \end{aligned}$$

Note that polynomial multiplication under secret sharing is not closed: the product has higher degree and thus more coefficients, potentially leaking the multiplicative depth of the computation.

However, the PSI protocol proposed in this paper has a particular structure: the RIE encoding only requires one multiplication (specifically, multiplication of depth 1) and several additionsan openly known constraint. Under such a restricted setting, Beaver triples can be naturally extended to work over $\mathbb{Z}_q[x]$.

5.3.3 Compatibility with Fast Fourier Transform

As discussed in the previous subsection, we propose a new approach that directly generalizes the original Beaver triple to the polynomial setting. Specifically, for any $a, b \in \mathbb{Z}_q^{\leq n}[x]$, we define:

$$[ab]_1, [ab]_2 \leftarrow \text{ABeaver}([a], [b])$$

This abstraction serves as a polynomial version of the multiplication protocol, denoted by $\text{Amul} := \text{ABeaver}$.

A major advantage of this generalized formulation is its natural compatibility with the Fast Fourier Transform (FFT)-based multiplication technique introduced in this paper. This integration significantly improves the efficiency of polynomial multiplication under secret sharing, reducing the computational complexity from $\Omega(n^2)$ to $\Omega(n \log n)$.

6 Updatable PSI

The *Updatable Private Set Intersection* problem (Updatable PSI [BMX22], UPSI) is a variant of the traditional *Private Set Intersection* problem (PSI [FNP04, HEK12]). It raises a new question: if the computing parties are allowed to update their sets over time, can efficient PSI protocols still be achieved?

Assume that the two parties involved in the Updatable PSI problem are denoted as $\mathcal{A}$ and $\mathcal{B}$. Initially, $\mathcal{A}$ holds a set A , and $\mathcal{B}$ holds a set B , with their intersection being $A \cap B$. Later, $\mathcal{A}$ adds several new elements to her set, forming a new subset Δ_A such that

$A \cap \Delta_A = \emptyset$ and $A' = A \cup \Delta_A$. Similarly, $\mathcal{B}$ adds a new subset Δ_B satisfying $B \cap \Delta_B = \emptyset$ and $B' = B \cup \Delta_B$. The updated intersection after both updates is therefore $A' \cap B'$.

In this work, we address the UPSI problem in multi-party settings by introducing a cloud-assisted architecture. To the best of our knowledge, UPSI in the multi-party scenario remains an open problem. Existing works [ATD20] mainly focus on efficiently updating each users own set, but they do not address the more critical challenge of efficiently computing the intersection over updated sets. Our work takes a step forward by filling this gap, providing the first practical and complete solution for multi-party UPSI.

6.1 Intuition

The intuition behind our UPSI construction is based on the following observation. Suppose one of the parties, say P_i , wants to delete an element from its set. Since the hash-to-bins technique has been used to partition each party's set into h bins, P_i only needs to update the bin containing the element to be deleted, say the j -th bin. Then, all parties collaboratively perform a multi-party PSI protocol on the collection of all parties j -th bins, thereby efficiently computing the updated intersection.

However, this approach introduces a privacy concern: revealing which bin is being updated (in this case, the j -th bin) may leak sensitive information, as the bin index j is closely correlated with the deleted element and may expose it.

To summarize, the key challenge becomes: *Can we update a specific bin without revealing which element triggered the update, thereby preserving privacy?* This can be achieved by introducing an Oblivious Pseudorandom Function (OPRF), whose ideal functionality is defined as follows:

1. Two parties are involved: P_1 and P_2 .
2. **Input:** P_1 holds a secret input x , and P_2 holds a secret key k .
3. **Output:** P_1 learns $\text{PRF}_k(x)$ but not k , while P_2 learns nothing about x .

In our protocol, the cloud holds the secret key k and performs OPRF with every party P_i to randomize their set elements, i.e., for each $a_i \in A_i$, the cloud and P_i jointly compute $\text{PRF}_k(a_i)$ such that P_i obtains $\text{PRF}_k(a_i)$ but the cloud learns nothing about a_i .

This OPRF-based preprocessing eliminates the information leakage issue mentioned above: even if P_i reveals which bin is updated in the UPSI process, this information becomes pseudorandom and independent of the deleted element, as long as the cloud does not reveal its PRF key k .

6.2 New constructions

By introducing a cloud server, we can construct an Updatable PSI (UPSI) protocol based on the OPRF primitive. Specifically, the protocol consists of three phases as follows.

6.2.1 Initialization Phase

In the initialization phase, all computing parties $\{P_i\}$ perform a distributed OPRF preprocessing over their sets $\{A_i\}$. This initialization only needs to be executed once throughout the entire UPSI protocol lifecycle.

The initialization phase proceeds as follows:

1. For each party P_i , the cloud $\mathcal{C}$ and P_i run the OPRF protocol to obtain the randomized set $A'_i = \{\text{PRF}_k(a_i) \mid a_i \in A_i\}$.
2. Each party applies the hash-to-bins technique to partition its randomized set A'_i into h bins.

3. All parties then jointly perform h instances of the multi-party PSI protocol each instance computes the intersection among all parties j -th bins, for $j = 1, \dots, h$.

Remarks. After the initialization, each party's set is randomized via the OPRF process. The key property here is that all computing parties (except the cloud $\mathcal{C}$) do not know the PRF key k , yet they can still obtain the OPRF outputs. Therefore, the randomized sets can be safely shared among the computing parties (but not with the cloud) without leaking any private information. Each party P_i also maintains a local mapping between its original elements and their randomized counterparts, i.e.,

$$a_i \leftrightarrow \text{PRF}_k(a_i), \quad \forall a_i \in A_i.$$

6.2.2 Update Phase

We assume each party's set size is $|A_i| = n$. The update phase handles two cases: adding new elements and deleting existing ones.

Adding an element. Without loss of generality, suppose party P_i adds a new element a_{new} :

1. P_i and the cloud jointly run the OPRF protocol to obtain $\text{PRF}_k(a_{\text{new}})$.
2. P_i updates its randomized set as $A'_i \leftarrow A'_i \cup \{\text{PRF}_k(a_{\text{new}})\}$.

Deleting an element. Similarly, suppose P_i wants to delete an element a_{del} :

1. P_i locates the corresponding PRF value $\text{PRF}_k(a_{\text{del}})$ in its randomized set.
2. P_i updates its set as $A'_i \leftarrow A'_i \setminus \{\text{PRF}_k(a_{\text{del}})\}$.

Intersection Phase

After the update phase, suppose that party P_i has updated Δ_i bins. Let the set of indices of these updated bins be denoted as $\mathcal{B}_i^{\text{update}}$. The intersection phase is described as follows:

1. All parties select the corresponding Δ_i bins $\mathcal{B}_i^{\text{update}}$ and denote them as $U_1, \dots, U_N$. They then compute the intersection using a multi-party PSI protocol (e.g., our delegated multi-party PSI), and denote the result as

$$I_U = \bigcap_i U_i.$$

2. All other $(n - \Delta_i)$ bins remain unchanged and are denoted as $\bar{U}_1, \dots, \bar{U}_N$. Their intersection is

$$I_{\bar{U}} = \bigcap_i \bar{U}_i.$$

3. All parties finally obtain the updated intersection as

$$I_U \cup I_{\bar{U}}.$$

Remarks. We analyze the performance of the intersection phase. The performance bottleneck occurs in Step 1, where each bin contains $O(\log n)$ elements. Therefore, the total communication cost is

$$O(N \cdot \Delta_i \cdot \log n),$$

and the total computation cost is

$$O(N \cdot \Delta_i \cdot \log n \cdot \log \log n).$$

7 Malicious Security

In this section, we discuss the potential of enhancing our PSI framework to achieve *malicious security* by adapting the SPDZ paradigm, in which any party that deviates from the prescribed protocol can be detected, and any incorrect PSI result arising from such misbehavior will cause the protocol to abort.

It is worth noting that the standard SPDZ framework is originally defined over a finite field $\mathbb{F}_q$, and can be extended to certain ring settings such as $\mathbb{Z}_{2^k}$. However, the PSI protocol proposed in this work operates over the polynomial ring $\mathbb{Z}_q[x]$.

A key challenge arises because multiplication in $\mathbb{Z}_q[x]$ is not closed under a fixed degree bound: the product of two degree- n polynomials results in a polynomial of degree up to $2n$. This degree expansion increases the number of coefficients and thereby prevents the direct application of the standard SPDZ framework.

However, the PSI protocol presented here has a unique feature: it only requires a single multiplication (of multiplicative depth 1) and a few additions. Under such restricted conditions, the SPDZ framework can be extended to work over $\mathbb{Z}_q[x]$.

7.1 Overview of SPDZ for $\mathbb{Z}_q[x]$: Offline Phase

Before entering the online computation phase, the SPDZ protocol requires a preprocessing stage referred to as the *offline phase* $\Pi_{\text{SPDZ.offline}}$ to generate correlated randomness in secret-shared form. This includes secret shares of MAC keys, random values, and Beaver multiplication triples.

Abstractly, N parties jointly execute the following protocol:

$$\{\beta_i\}_{i=1}^N, [\alpha], \langle r \rangle, [r], \langle a \rangle, \langle b \rangle, \langle ab \rangle \leftarrow \Pi_{\text{SPDZ.offline}}()$$

where:

- β_i is the local MAC key assigned to party P_i ;
- $[\cdot]$ denotes *Type-II* secret sharing;
- $\langle \cdot \rangle$ denotes *Type-I* secret sharing;
- α is the global MAC key;
- r is a random value;
- (a, b, ab) forms a Beaver multiplication triple.

Each Type-I share of a value r held by party P_i is represented as:

$$\langle r \rangle_i := (r_i, \gamma(r)_i, \delta)$$

where $r = \sum_i r_i$, $\delta \in \mathbb{Z}_q$ is a global offset, and the MAC shares satisfy:

$$\sum_i \gamma(r)_i = \alpha(r + \delta)$$

Each Type-II share of r held by P_i is denoted as:

$$[r]_i := \left(r_i, \left\{ \gamma(r)_i^j \right\}_{j=1}^N \right)$$

Here, $r = \sum_i r_i$ as before, and the local MAC tags satisfy:

$$\sum_i \gamma(r)_i^j = \beta_j \cdot r \quad \text{for all } j \in [N]$$

The two secret-sharing types differ in their verification mechanisms. Type-I shares use a global MAC key and a single offset, while Type-II shares distribute MAC verifiability across all parties using their individual MAC keys. This dual-sharing structure enables SPDZ to support both efficient local computation and robust global verification in the online phase.

7.2 Online Phase of SPDZ

The *online phase* of SPDZ consists of three types of operations: input sharing, addition, and multiplication. Assume a set of N parties $\{P_i\}$ participate in the computation.

Input Sharing. When party P_i wishes to input a private value x , it secret-shares x among all parties using fresh randomness:

$$\{\langle x \rangle_j\}_{j=1}^N \leftarrow \Pi_{\text{SPDZ.share}}(x, \langle r \rangle, [r])$$

Each input sharing consumes a fresh pair of Type-I and Type-II shares of a random value r . For simplicity, this may be written as:

$$\{\langle x \rangle_j\}_{j=1}^N \leftarrow \Pi_{\text{SPDZ.share}}(x)$$

The sharing protocol proceeds as follows:

1. Each party P_j ($j \neq i$) sends its share $[r]_j$ to P_i
2. Party P_i reconstructs the value r using $\{[r]_j\}_j$
3. P_i sets its own share as:

$$\langle x \rangle_i \leftarrow x - r + \langle r \rangle_i$$

4. For all $j \neq i$, set $\langle x \rangle_j \leftarrow \langle r \rangle_j$

Addition. Addition is local and non-interactive. Given shares $\langle x \rangle_i = (x_i, \gamma(x)_i, \delta_x)$ and $\langle y \rangle_i = (y_i, \gamma(y)_i, \delta_y)$, party P_i computes:

$$\langle x + y \rangle_i \leftarrow (x_i + y_i, \gamma(x)_i + \gamma(y)_i, \delta_x + \delta_y)$$

Multiplication. Multiplication is more complex and involves interaction and preprocessed Beaver triples $\langle a \rangle, \langle b \rangle, \langle ab \rangle$. Assuming all parties hold secret shares $\langle x \rangle_i$ and $\langle y \rangle_i$, the multiplication is denoted as:

$$\langle xy \rangle_i \leftarrow \Pi_{\text{SPDZ.mul}}(\langle x \rangle, \langle y \rangle, \langle a \rangle, \langle b \rangle, \langle ab \rangle)$$

or simply:

$$\langle xy \rangle_i \leftarrow \langle x \rangle_i \cdot \langle y \rangle_i$$

The multiplication protocol proceeds as follows:

1. Each party computes:

$$\langle \epsilon \rangle_i \leftarrow \langle x \rangle_i - \langle a \rangle_i, \quad \langle \delta \rangle_i \leftarrow \langle y \rangle_i - \langle b \rangle_i$$

2. Parties jointly reconstruct $\epsilon = x - a$ and $\delta = y - b$ using partial openings (i.e., without MAC checking).
3. Party P_1 computes:

$$\langle xy \rangle_1 \leftarrow \epsilon \cdot \delta + \epsilon \cdot \langle b \rangle_1 + \delta \cdot \langle a \rangle_1 + \langle ab \rangle_1$$

4. Each party P_i ($i \neq 1$) computes:

$$\langle xy \rangle_i \leftarrow \epsilon \cdot \langle b \rangle_i + \delta \cdot \langle a \rangle_i + \langle ab \rangle_i$$

5. After multiplication, all parties perform *delayed MAC checking* on $\langle \epsilon \rangle_i$ and $\langle \delta \rangle_i$ to ensure integrity. This is possible once the global MAC key α is revealed at the end of the computation.

Partial Openings. The partial opening in Step 2 reveals the plaintext value ϵ or δ , but does not verify its MAC tags. This avoids prematurely revealing or relying on the global MAC key α , which is withheld until the final verification phase. If any malicious behavior is detected during MAC checks, the computation is aborted.

7.3 Malicious security framework

Finally, we describe the overall framework of the protocol using the extended SPDZ model over the polynomial ring $\mathbb{Z}_q[x]$ as shown in Protocol 4.

8 Conclusion

In this work, we redefine the Delegated PSI problem, emphasizing the importance of silent processing and minimal user-to-user communication as critical requirements for real-world cloud computing scenarios. We propose an ideal construction for the Delegated PSI protocol, demonstrating that the best achievable computational and communication complexities are both $\Omega(Nk)$, where N is the number of users in the protocol and k is the size of each user's set.

We first present a baseline construction based on randomized intersection encoding and additive secret sharing, augmented by Beaver's multiplication trick. This baseline construction achieves $\Omega(Nk^2)$ computational overhead and optimal $\Omega(Nk)$ communication overhead. We then introduce a computationally optimized construction that reduces the computational overhead to $\tilde{\Omega}(Nk)$, at the cost of increasing the communication complexity to $\tilde{\Omega}(Nk)$. The optimization techniques employed include hash table representation of a set and fast fourier transformation of a polynomial. However, we argue that the optimized construction achieves near-optimal performance compared to the ideal construction, ignoring logarithmic factors in the asymptotic complexity analysis.

Additionally, we explore how to efficiently support delegated PSI in dynamic settings, where users may frequently update their delegated sets stored on the cloud. We also address the challenge of upgrading semi-honest delegated PSI to malicious security by leveraging the widely adopted SPDZ framework.

We are currently implementing the protocols proposed in this work to provide experimental evidence demonstrating that these protocols are not only asymptotically efficient but also concretely practical. Upon completing our experiments, we will release the source code and update this paper with the corresponding experimental results.

References

- [ATD15] Aydin Abadi, Sotirios Terzis, and Changyu Dong. O-psi: delegated private set intersection on outsourced datasets. In *ICT Systems Security and Privacy Protection: 30th IFIP TC 11 International Conference, SEC 2015, Hamburg, Germany, May 26-28, 2015, Proceedings 30*, pages 3–17. Springer, 2015.

Delegated PSI Protocol with Malicious Security DPSI-SPDZ protocol	
Input	A semi-honest cloud C ; N computing parties $\{P_1, \dots, P_N\}$.
Output	A designated evaluator P_{N+1} learns the intersection $\bigcap_{i=1}^N A_i$.
1	Each party P_i re-encodes its private set by appending a hash: $a \leftarrow a \parallel h(a), \quad \forall a \in A_i$
2	Each P_i encodes its set A_i as a polynomial: $p_i(x) = \prod_{a \in A_i} (x - a)$
3	Each P_i generates a Type-I secret sharing of p_i : $\langle p_i \rangle = (\langle p_i \rangle_0, \langle p_i \rangle_1)$ and sends $\langle p_i \rangle_0$ to C while retaining $\langle p_i \rangle_1$.
4	Cloud C samples random polynomials $\omega_1, \dots, \omega_N \in \mathbb{Z}_q[x]$, and secret-shares them: $\langle \omega_i \rangle = (\langle \omega_i \rangle_0, \langle \omega_i \rangle_1)$ sending $\langle \omega_i \rangle_1$ to P_i and keeping $\langle \omega_i \rangle_0$.
5	Cloud C computes the weighted sum share: $\left\langle \sum_{i=1}^N \omega_i p_i \right\rangle_0 = \sum_{i=1}^N \langle \omega_i \rangle_0 \cdot \langle p_i \rangle_0$
6	Each party P_i locally computes: $\langle \omega_i p_i \rangle_1 = \langle \omega_i \rangle_1 \cdot \langle p_i \rangle_1$
7	Cloud C sends $\langle \sum_i \omega_i p_i \rangle_0$ to the designated evaluator P_{N+1} via a secure channel.
8	Each P_i sends $\langle \omega_i p_i \rangle_1$ to P_{N+1} via a secure channel.
9	Evaluator P_{N+1} reconstructs the polynomial: $r(x) = \sum_{i=1}^N \omega_i(x) \cdot p_i(x)$ and determines the intersection $\bigcap_i A_i$ from the roots of $r(x)$.

Figure 4: Delegated PSI protocol with malicious security using SPDZ over $\mathbb{Z}_q[x]$

- [ATD17] Aydin Abadi, Sotirios Terzis, and Changyu Dong. Vd-psi: verifiable delegated private set intersection on outsourced private datasets. In *Financial Cryptography and Data Security: 20th International Conference, FC 2016, Christ Church, Barbados, February 22–26, 2016, Revised Selected Papers 20*, pages 149–168. Springer, 2017.
- [ATD20] Aydin Abadi, Sotirios Terzis, and Changyu Dong. Feather: Lightweight multi-

- party updatable delegated private set intersection. *Cryptology ePrint Archive*, 2020.
- [ATMD17] Aydin Abadi, Sotirios Terzis, Roberto Metere, and Changyu Dong. Efficient delegated private set intersection on outsourced private datasets. *IEEE Transactions on Dependable and Secure Computing*, 16(4):608–624, 2017.
 - [BEKSS22] Aner Ben-Efraim, Andras Kiss, Thomas Schneider, and Abhi Shelat. PSimple: Practical multiparty maliciously-secure private set intersection. In *Proceedings of the 2022 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pages 595–608. ACM, 2022.
 - [BMX22] Saikrishna Badrinarayanan, Peihan Miao, and Tiancheng Xie. Updatable private set intersection. *Proceedings on Privacy Enhancing Technologies*, 2022(2), 2022.
 - [CGST21] Nishanth Chandran, Divya Gupta, Akshay Shah, and Divyashree Tripathy. Efficient linear multiparty psi and extensions to circuit/quorum psi. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2759–2776. ACM, 2021.
 - [FNP04] Michael J Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In *International conference on the theory and applications of cryptographic techniques*, pages 1–19. Springer, 2004.
 - [GLWS24] Ying Gao, Jian Liu, Xiao Wang, and Elaine Shi. Efficient scalable multi-party private set intersection (-variants) from bicentric zero-sharing. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2024.
 - [GPR⁺21] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious key-value stores and amplification for private set intersection. In *Annual International Cryptology Conference*, pages 395–425. Springer, 2021.
 - [GZWS25] Yewei Guan, Jian Zhang, Xiao Wang, and Elaine Shi. Practical multiparty private set intersection with reducible zero-sharing. *Cryptology ePrint Archive*, Paper 2025/XXX, 2025. <https://eprint.iacr.org/2025/XXX>.
 - [HEK12] Yan Huang, David Evans, and Jonathan Katz. Private set intersection: Are garbled circuits better than custom protocols? In *NDSS*, 2012.
 - [HFH99] Bernardo A Huberman, Matt Franklin, and Tad Hogg. Enhancing privacy and trust in electronic communities. In *Proceedings of the 1st ACM conference on Electronic commerce*, pages 78–86, 1999.
 - [Ker12] Florian Kerschbaum. Outsourced private set intersection using homomorphic encryption. In *Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security*, pages 85–86, 2012.
 - [KKRT17] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1257–1272. ACM, 2017.

- [KMRS14] Seny Kamara, Payman Mohassel, Mariana Raykova, and Saeed Sadeghian. Scaling private set intersection to billion-element sets. In *Financial Cryptography and Data Security: 18th International Conference, FC 2014, Christ Church, Barbados, March 3-7, 2014, Revised Selected Papers 18*, pages 195–215. Springer, 2014.
- [KS05] Lea Kissner and Dawn Song. Privacy-preserving set operations. In *Annual International Cryptology Conference*, pages 241–257. Springer, 2005.
- [LNZ⁺14] Fang Liu, Wee Keong Ng, Wei Zhang, Shuguo Han, et al. Encrypted set intersection protocol for outsourced datasets. In *2014 IEEE International Conference on Cloud Engineering*, pages 135–140. IEEE, 2014.
- [Mea86] Catherine Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In *1986 IEEE Symposium on Security and Privacy*, pages 134–134. IEEE, 1986.
- [NTY21] Ofri Nevo, Ni Trieu, and Avishay Yanai. Simple, fast malicious multiparty private set intersection. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2119–2135. ACM, 2021.
- [QLS⁺15] Shuo Qiu, Jiqiang Liu, Yanfeng Shi, Ming Li, and Wei Wang. Identity-based private matching over outsourced encrypted datasets. *IEEE Transactions on cloud Computing*, 6(3):747–759, 2015.
- [RS21] Peter Rindal and Phillipp Schoppmann. Vole-psi: fast oprf and circuit-psi from vector-ole. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 901–930. Springer, 2021.
- [WYC24] Mingli Wu, Tsz Hon Yuen, and Kwan Yin Chan. O-Ring and K-Star: Efficient multi-party private set intersection. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024.
- [YY25] Foo Yee Yeo and Jason H. M. Ying. Private set intersection and other set operations in the third party setting. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025.
- [ZX15] Qingji Zheng and Shouhuai Xu. Verifiable delegated set intersection operations on outsourced encrypted data. In *2015 IEEE International Conference on Cloud Engineering*, pages 175–184. IEEE, 2015.